

Software Development Fundamentals (SDF) – I

ODD 2023



Lecture 7

Practice Questions on Operators:

1. Write a program to swap two numbers without using a temporary variable. (Hint: Use arithmetic operators)
2. Write a program to check if a number is a power of 2 using bitwise operators.
3. Write a program to find the smallest of three numbers using the conditional operator.
4. Write a program to check if the given number is perfect.
5. Write a program to print if a number is even or odd without using any arithmetic or relational operator.
6. Write a program that multiplies a number by 8 using bitwise operators.
7. Given the code snippet, what will it print?
8. Set n-th bit of a number using bitwise operators
9. Clear n-th bit of a number using bitwise operators
10. Toggle n-th bit of a number using bitwise operators
11. Check if a number is negative using bitwise operators

1. **Solution using Arithmetic Operators:**

```
#include <stdio.h>

int main() {
    int a = 5, b = 7;

    printf("Before swapping: a = %d, b = %d\n", a, b);

    a = a + b;
    b = a - b; // This now becomes (a + b) - b = a
    a = a - b; // This now becomes (a + b) - a = b

    printf("After swapping: a = %d, b = %d\n", a, b);

    return 0;
}
```

Solution using Bitwise Operators:

```
#include <stdio.h>

int main() {
    int a = 5, b = 7;

    printf("Before swapping: a = %d, b = %d\n", a, b);

    a = a ^ b;
    b = a ^ b; // This now becomes (a ^ b) ^ b = a
    a = a ^ b; // This now becomes (a ^ b) ^ a = b

    printf("After swapping: a = %d, b = %d\n", a, b);

    return 0;
}
```

2.

```
#include <stdio.h>

int main() {
    int num = 16;

    // A number is a power of 2 if it has only one '1' bit in binary representation.
    // num & (num - 1) will be zero for power of 2 numbers.
    if (num > 0 && (num & (num - 1)) == 0) {
        printf("%d is a power of 2.\n", num);
    } else {
        printf("%d is not a power of 2.\n", num);
    }

    return 0;
}
```

3.

```
#include <stdio.h>

int main() {
    int a = 10, b = 5, c = 8;

    int smallest = (a < b) ? ((a < c) ? a : c) : ((b < c) ? b : c);

    printf("The smallest number among %d, %d, and %d is: %d\n", a, b, c, smallest);

    return 0;
}
```

4. A perfect number is a positive integer that is equal to the sum of its proper divisors, excluding itself. For example, 28 is a perfect number because its divisors are 1, 2, 4, 7, 14, and the sum of these values is 28. Write a program to check if the given number is perfect.

```
#include <stdio.h>
int main() {
    int num, sum = 0;
    printf("Enter a number: ");
    scanf("%d", &num);

    for (int i = 1; i <= num / 2; i++) {
        if (num % i == 0) {
            sum += i;
        }
    }
}
```

```
if (sum == num) {
    printf("%d is a perfect
number.\n", num);
} else {
    printf("%d is not a perfect
number.\n", num);
}

return 0;
}
```

5. Write a program to print if a number is even or odd without using any arithmetic or relational operator.

```
#include <stdio.h>

int main() {
    int num = 4;

    (num & 1) ? printf("Odd\n") : printf("Even\n");

    return 0;
}
```

The bitwise & operator is used to check the last bit of the integer. If it's 1, the number is odd, otherwise even.

6. Write a program that multiplies a number by 8 using bitwise operators.

```
#include <stdio.h>
```

```
int main() {
```

```
    int num = 4;
```

```
    int result = num << 3;
```

```
    printf("Number after multiplication by 8: %d\n", result);
```

```
    return 0;
```

```
}
```

Note: Left shifting a number by n bits means multiplying it by 2^n . So, left shifting by 3 bits multiplies the number by 8.

7. Given the following code snippet, what will it print?

```
int main() {  
    int i = 1, j = 2, k = 3;  
    printf("%d", (i, j, k));  
    return 0;  
}
```

Solution:

The comma operator evaluates its first operand and discards the result, then evaluates the second operand and returns its value. In the given code, i, j, and k are evaluated in order, but only the value of k (which is 3) is returned and printed. So, the program will print 3.

8. Set n-th bit of a number

```
#include <stdio.h>
```

```
int main() {
```

```
    int num, n;
```

```
    printf("Enter an integer: ");
```

```
    scanf("%d", &num);
```

```
    printf("Enter the bit position (0-based) to set: ");
```

```
    scanf("%d", &n);
```

```
    num |= (1 << n);
```

```
    printf("Number after setting %d-th bit: %d\n", n, num);
```

```
    return 0;
```

```
}
```

The line `num |= (1 << n);` sets the n-th bit to 1. `1 << n` shifts the bit 1 to the left by n places. The `|=` operator then performs a bitwise OR operation with the original number and the shifted value, which effectively sets the n-th bit.

9. Clear n-th bit of a number

```
#include <stdio.h>
```

```
int main() {
```

```
    int num, n;
```

```
    printf("Enter an integer: ");
```

```
    scanf("%d", &num);
```

```
    printf("Enter the bit position (0-based) to clear: ");
```

```
    scanf("%d", &n);
```

```
    num &= ~(1 << n);
```

```
    printf("Number after clearing %d-th bit: %d\n", n, num);
```

```
    return 0;
```

```
}
```

This program clears (sets to 0) the n-th bit of an integer. The line **num &= ~(1 << n);** clears the n-th bit.

1 << n again shifts the bit 1 to the left by n places.

~(1 << n) then inverts all the bits, making our desired bit 0 and all others 1.

The &= operator performs a bitwise AND operation with the original number and the inverted shifted value, which clears the n-th bit.

10. Toggle n-th bit of a number

```
#include <stdio.h>
```

```
int main() {
```

```
    int num, n;
```

```
    printf("Enter an integer: ");
```

```
    scanf("%d", &num);
```

```
    printf("Enter the bit position (0-based) to toggle: ");
```

```
    scanf("%d", &n);
```

```
    num ^= (1 << n);
```

```
    printf("Number after toggling %d-th bit: %d\n", n, num);
```

```
    return 0;
```

```
}
```

This program toggles the n-th bit of an integer. If it's 1, it becomes 0, and vice-versa.

The line **num ^= (1 << n);** toggles the n-th bit.

1 << n does the usual left shift.

The ^= operator is a bitwise XOR operation. If the n-th bit in the original number is 1, it'll be set to 0, and if it's 0, it'll be set to 1.

11. Check if a number is negative using bitwise operators

```
#include <stdio.h>
```

```
int main() {  
    int num;  
    printf("Enter an integer: ");  
    scanf("%d", &num);  
    if (num & (1 << (sizeof(int) * 8 - 1))) {  
        printf("The number is negative.\n");  
    } else {  
        printf("The number is positive or zero.\n");  
    }  
  
    return 0;  
}
```

This program checks if a given integer is negative by examining its most significant bit (the sign bit in Two's complement representation).

In two's complement representation, the most significant bit (MSB) represents the sign of the number. If it's 1, the number is negative; if it's 0, the number is non-negative.

`sizeof(int) * 8 - 1` calculates the position of the MSB (0-based).

The line `if (num & (1 << (sizeof(int) * 8 - 1)))` checks if the MSB is 1.

**End of
Lecture 7**